

DIRECTIVES D'INTÉGRATION DU SYSTÈME DE COMBAT

PROFIL DE PROTECTION DE L'INTERFACE HTML/CSS EXISTANTE



DIRECTIVE CRITIQUE POUR L'IA DE DÉVELOPPEMENT

INTERDICTION STRICTE de modifier la structure DOM, l'arborescence HTML, les classes CSS (Aurora Forge, tokens, styles) ou l'agencement visuel de l'application. Ton rôle est uniquement d'injecter la logique JavaScript dans les ID existants et de manipuler dynamiquement les jauges fournies sans altérer le design.

1. Cartographie du DOM Réel (index.html)

Le script logique de combat doit se greffer exclusivement sur les nœuds HTML existants décrits ci-dessous :

Mécanique Spécifiée	ID / Sélecteur HTML Cible	Action Logique attendue
Vie Globale Équipe	#party-hp-bar-fill #party-hp-text	Modifier le style width (%) et mettre à jour le texte textContent ("Actuel / Max").
Brave Burst Slots	#footer-bb-slots	Injecter dynamiquement les boutons/jauges individuelles des héros de la squad active.
Nom & Vie Ennemi	#monster-name #hp-bar-fill #hp-text	Mettre à jour le nom de la cible et synchroniser sa jauge de PV lors des clics.
Timer du Boss	#boss-timer #boss-label	Gérer le compte à rebours de 30s. Si à 0, échec du stage (retour au Hub).
Mode Auto	#combat-auto-btn	Écouter le clic pour basculer l'état automatique (génération fixe sans bonus).

2. Algorithme Applicatif Révisé

Le calcul des dégâts doit s'interfacer avec le système de scaling présent dans `./js/game.js` sans casser les multiplicateurs de zone existants.

Gestion des dégâts et de la "Réfraction"

Lors d'une attaque ennemie, au lieu d'impacter des entités physiques distinctes, le script doit cibler un index (0 à 3) correspondant à l'un des héros présents dans le composant `#squad-grid`.

```
// Exemple d'injection logique préservant l'UI
function appliquerDegatsEnnemi(degatsBruts, elementEnnemi) {
  const squadActive = gameData.heroes.filter(h => h.inSquad);
  if(squadActive.length === 0) return;

  // Choix aléatoire du slot ciblé parmi les héros présents
  const targetIndex = Math.floor(Math.random() * squadActive.length);
  const heroCible = squadActive[targetIndex];

  // Calcul avec le multiplicateur élémentaire
  const mult = obtenirMultiplicateurElementaire(elementEnnemi, heroCible.element);
  const degatsFinaux = Math.max(0, (degatsBruts * mult) - heroCible.stats.def);

  // Impact sur la jauge de vie commune
  combatState.teamGlobalHP = Math.max(0, combatState.teamGlobalHP - degatsFinaux);

  // Rendu visuel strict sans altérer la structure
  document.getElementById('party-hp-bar-fill').style.width = `${
    (combatState.teamGlobalHP / combatState.teamMaxGlobalHP) * 100}%`;
  document.getElementById('party-hp-text').textContent = `${combatState.teamGlobalHP} / ${
    combatState.teamMaxGlobalHP}`;
}
```

3. Règles de Chargement Déterministe du Brave Burst (BB)

- La jauge maximale de chaque héros généré dans `#footer-bb-slots` est fixée à **100 points**.
- **En mode AUTO ON** : Chaque héros accumule automatiquement +10 BC (Brave Crystals) par tour de table de manière brute et linéaire.
- **En mode MANUAL (AUTO OFF)** : Si le joueur clique sur l'arène de combat (`#monster-zone`), cela génère des points de soutien (actuellement configurés visuellement à +3 BC dans ton en-tête via `#tap-dmg-val`). Ces points doivent être répartis ou accumulés pour accélérer le déclenchement des compétences.

4. Préservation des Scripts Annexes

Le code produit doit cohabiter proprement avec les bundles et modules déclarés en fin de fichier :

- `./assets/globals.bundle.js` (qui gère la sérialisation/formatage des nombres).
- `./js/ui-shell.js` (responsable des transitions d'onglets et du tiroir coulissant).
- `./js/start-menu.js` (gérant l'initialisation de la partie).